Inteligență Artificială

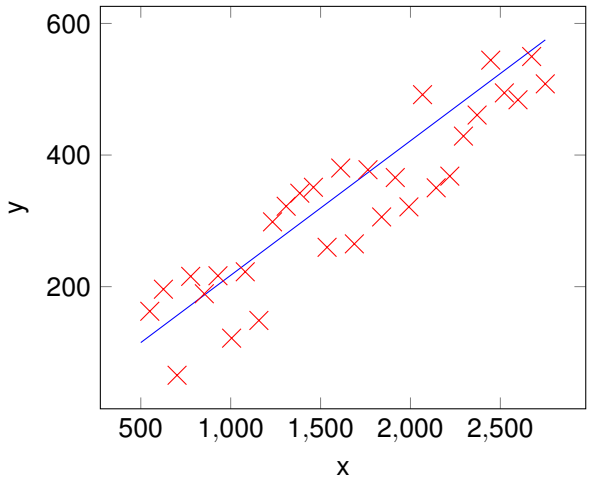
Regresia Liniară

Slides: Mihai Trăscău

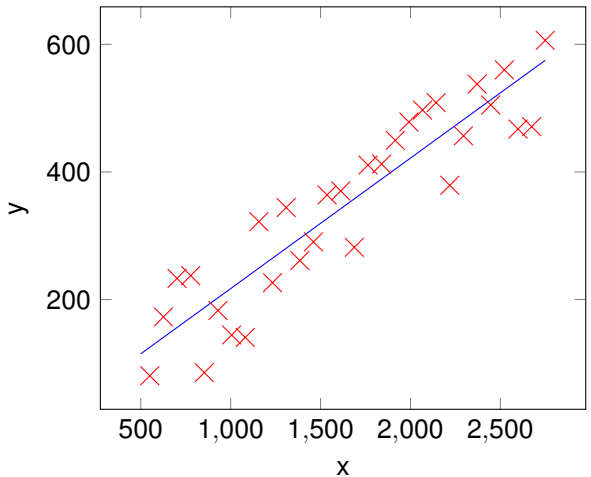
- Regresia Liniară
- Arbori de regresie

- Fie setul de învățare cu n exemple, $\mathcal{X} = \{X^{(1)}, X^{(2)}, ..., X^{(n)}\}$
- Fiecare exemplu este descris de m atribute: $X^{(i)} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$
- Funcția f indică valoarea țintă adevărată (eng. *ground truth*) pentru fiecare $X^{(i)}$
- Căutăm ipoteza h din spațiul de ipoteze posibile H pentru care $h(X^{(i)}) = f(X^{(i)})$ pentru $\forall i$ cu $i = \overline{1, n}$
- Pentru clasificare f returnează valori discrete $\Rightarrow$ grupează exemple
- Pentru regresie f returnează valori continue $\Rightarrow$ estimează/prezice valori

- Regresia liniară este un algoritm de învățare supervizată
- Învață să prezică valori continue pe baza exemplelor de învățare



- Cum se reprezintă ipoteza h?
- În exemplu avem o regresie liniară cu 1 variabilă (valoarea lui y este dată de x)
- Modelăm ipoteza folosind 2 parametri, θ_0, θ_1: $h_\theta(x) = \theta_0 + \theta_1 x$



- Notăm:
  - n – număr de exemple de învățare
  - x – variabila de intrare (atribut)
  - y – valoarea de ieșire din setul de date
  - θ_0, θ_1 – parametrii modelului
- Cum găsim θ_0 și θ_1 astfel încât $h_\theta(x)$ să corespundă cu y?
- Soluție: Minimizăm funcția de cost, eroare pătratică medie – MSE (eng. *Mean Squared Error*)

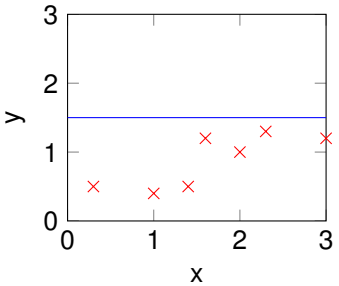
- Plecând de la $h_\theta(x) = \theta_0 + \theta_1 x$
- Și funcția de eroare pătratică medie (MSE):

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^{n} \left(h(x^{(i)}) - y^{(i)} \right)^2$$

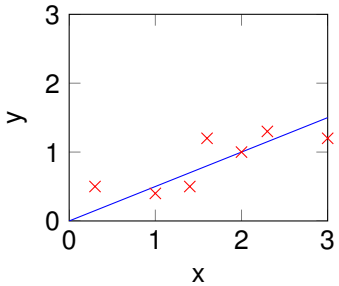
- Trebuie să minimizăm:

$$\underset{\theta_0, \theta_1}{\operatorname{argmin}} \, J(\theta_0, \theta_1)$$

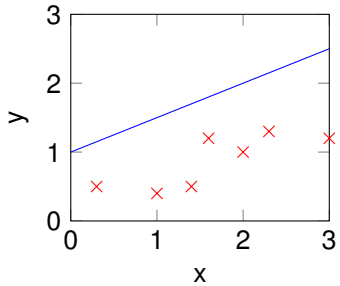
- Plecând de la $h_\theta(x) = \theta_0 + \theta_1 x$
- Trebuie să alegem parametrii θ_0 și θ_1 astfel încât $h_\theta(x)$ să corespundă cu y



$\theta_0 = 1.5$
$\theta_1 = 0$

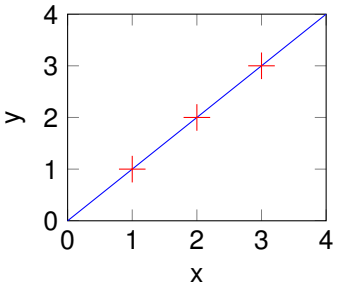


$\theta_0 = 0$
$\theta_1 = 0.5$

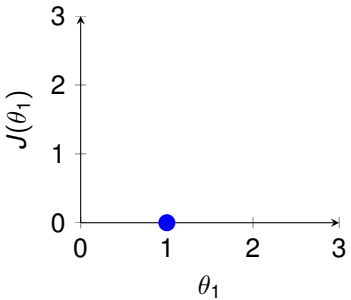


$\theta_0 = 1$
$\theta_1 = 0.5$

- Presupunem numai drepte cu $\theta_0 = 0$
- $h_\theta(x) = \theta_1 x$

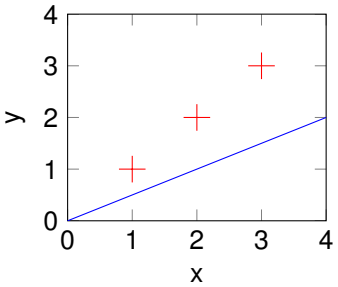


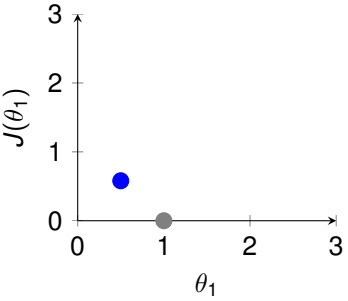
$\theta_1 = 1$



$J(1) = 0$

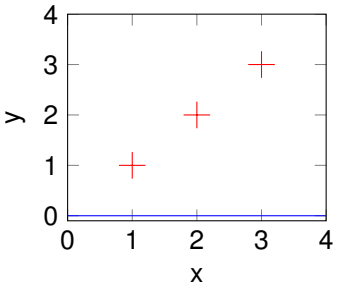
- Presupunem numai drepte cu $\theta_0 = 0$
- Testăm mai multe valori ale lui θ_1 pentru $h_\theta(x) = \theta_1 x$

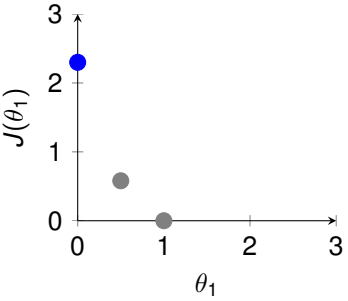




$\theta_1 = 0.5$ $J(0.5) \approx 0.58$

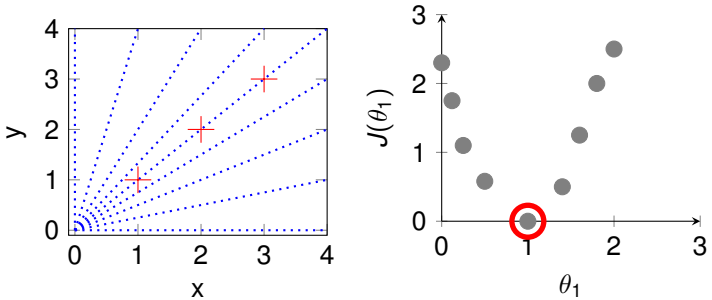
- Presupunem numai drepte cu $\theta_0 = 0$
- Testăm mai multe valori ale lui θ_1 pentru $h_\theta(x) = \theta_1 x$



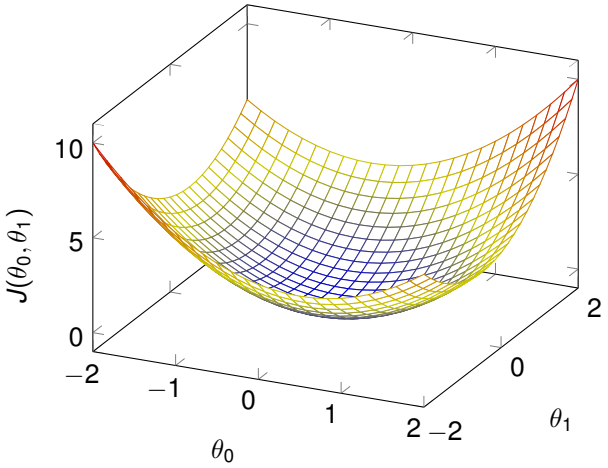


$\theta_1 = 0$ $J(0) \approx 2.3$

- Presupunem numai drepte cu $\theta_0 = 0$
- Testăm mai multe valori ale lui θ_1 pentru $h_\theta(x) = \theta_1 x$
- Pentru fiecare valoare θ_1 obținem o valoare diferită pentru $J(\theta_1)$
- Găsim că valoarea minimă a lui $J(\theta_1)$ este pentru $\theta_1 = 1$ (când $\theta_0 = 0$)



- Cum ar arăta o căutare simultană a lui θ_0 și θ_1?



- Cum putem extinde problema la mai multe variabile?
- Considerăm mai multe atribute pentru exemplele din setul de învățare $\mathbf{x}^{(i)} : [x_1, x_2, ..., x_m]$
- Astfel, ipoteza $h(\mathbf{x})$ devine:

$$h(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + ... + \theta_m x_m$$

- În scrierea vectorială considerăm un atribut suplimentar $x_0 = 1$ pentru orice exemplu:

$$X = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_m \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_m \end{bmatrix} \quad \text{deci ipoteza devine } h(x) = \theta^T x$$

- Funcția de cost corespunzătoare ipotezei $h(\mathbf{x})$ este $J(\theta_0, \theta_1, ..., \theta_m)$, de forma:

$$J(\theta_0, \theta_1, ..., \theta_m) = \frac{1}{2n} \sum_{i=1}^{n} \left(h(x^{(i)}) - y^{(i)} \right)^2$$

- Funcția de cost corespunzătoare ipotezei $h(\mathbf{x})$ este $J(\theta_0, \theta_1, ..., \theta_m)$, de forma:

$$J(\theta_0, \theta_1, ..., \theta_m) = \frac{1}{2n} \sum_{i=1}^{n} \left(h(x^{(i)}) - y^{(i)} \right)^2$$

- Observăm că putem exprima cele n ecuații de forma $h(\mathbf{x^i}) = y^i$ ca $h(\mathbf{x}) = \mathbf{X} \cdot \theta = \mathbf{y}$

$$\begin{bmatrix} x_0^1, x_1^1, \ldots, x_m^1 \\ x_0^2, x_1^2, \ldots, x_m^2 \\ \vdots, \vdots, \ldots, \vdots \\ x_0^n, x_1^n, \ldots, x_m^n \end{bmatrix} \times \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_m \end{bmatrix} = \begin{bmatrix} y^1 \\ y^2 \\ \vdots \\ y^n \end{bmatrix}$$

Astfel, funcția de cost se poate exprima ca:

$$J(\theta_0, \theta_1, ..., \theta_m) = \frac{1}{2n} (\mathbf{X}\theta - \mathbf{y})^T (\mathbf{X}\theta - \mathbf{y})$$

- Ce algoritmi avem la dispoziție pentru a găsi valorile $\theta_0, \theta_1, ..., \theta_m$?

- Ce algoritmi avem la dispoziție pentru a găsi valorile $\theta_0, \theta_1, ..., \theta_m$?
- Adică, cum găsim minimul funcției $J(\theta_0, \theta_1, ..., \theta_m)$?

- Ce algoritmi avem la dispoziție pentru a găsi valorile $\theta_0, \theta_1, ..., \theta_m$?
- Adică, cum găsim minimul funcției $J(\theta_0, \theta_1, ..., \theta_m)$?
- Soluție 1: Putem calcula o soluție analitică

- Ce algoritmi avem la dispoziție pentru a găsi valorile $\theta_0, \theta_1, ..., \theta_m$?
- Adică, cum găsim minimul funcției $J(\theta_0, \theta_1, ..., \theta_m)$?
- Soluție 1: Putem calcula o soluție analitică
- Soluție 2: Putem utiliza o metodă iterativă de optimizare

- Putem calcula o soluție analitică care găsește direct θ pentru care funcția $J(\theta_0, \theta_1, ..., \theta_m)$ este minimă
- Minimul unic al unei funcții strict convexe se găsește când:

$$\frac{\partial J(\theta)}{\partial \theta} = 0$$

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{\partial}{\partial \theta}\left(y^T y - 2y^T X\theta + \theta^T X^T X\theta\right)$$

$$\Rightarrow \boxed{\theta = (X^T X)^{-1} X^T y}$$

- Unde X reprezintă întreg setul de date de învățare

- Limitări ale soluției analitice:
  - Presupunerea strict convexă este simplificatoare (de obicei este încălcată)

- Limitări ale soluției analitice:
  - Presupunerea strict convexă este simplificatoare (de obicei este încălcată)
  - Dificultăți la calculul pseudo-inversei $(X^T X)^{-1}$ (complexitate de calcul $O(n^3)$)

- Limitări ale soluției analitice:
  - Presupunerea strict convexă este simplificatoare (de obicei este încălcată)
  - Dificultăți la calculul pseudo-inversei $(X^T X)^{-1}$ (complexitate de calcul $O(n^3)$)
  - Setul de date X poate fi foarte mare $\Rightarrow$ calculul devine complicat, chiar intractabil

- Limitări ale soluției analitice:
  - Presupunerea strict convexă este simplificatoare (de obicei este încălcată)
  - Dificultăți la calculul pseudo-inversei $(X^T X)^{-1}$ (complexitate de calcul $O(n^3)$)
  - Setul de date X poate fi foarte mare $\Rightarrow$ calculul devine complicat, chiar intractabil
  - Este dependentă de forma funcției de eroare. Pentru unele funcții (e.g. *Lasso*) nu există soluție analitică de formă închisă.

- Scăderea gradientului (eng. gradient descent) este o metodă iterativă pentru găsirea minimului unei funcții
- Plecând de la o configurație inițială (de exemplu, aleatoare), se fac pași proporționali cu negativul gradientului funcției în punctul curent calculat

Gradient-Descent($\Theta_{init}, \alpha, f, f', \epsilon$) – caută minimul funcției f

Intrări: Θ_{init} – valorile inițiale, α – rata de învățare
f – funcția de minimizat, f' – derivata funcției de minimizat
ϵ – prag pentru criteriul de convergență

1. $\Theta^{(0)} \leftarrow \Theta_{init}$
2. $t \leftarrow 0$
3. **repetă**
4. $\quad t \leftarrow t + 1$
5. $\quad \Theta^{(t)} = \Theta^{(t-1)} - \alpha f'(\Theta^{(t-1)})$
6. **până când** $|f(\Theta^{(t)}) - f(\Theta^{(t-1)})| < \epsilon$
7. **întoarce** $\Theta^{(t)}$

- Pentru regresia liniară cu funcție de eroare MSE, avem:

$$J(\theta_0, \theta_1, ..., \theta_m) = \frac{1}{2n} \sum_{i=1}^{n} \left(h(x^{(i)}) - y^{(i)} \right)^2$$

- Derivata funcției de eroare este:

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{n} \sum_{i=1}^{n} \left(h(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

Gradient-Descent-MSE($\Theta_{init}, \alpha, \epsilon$) – caută minimul funcției $J(\theta)$

Intrări: Θ_{init} – valorile inițiale
α – rata de învățare
ϵ – prag pentru criteriul de convergență

1. $\Theta^{(0)} \leftarrow \Theta_{init}$
2. $t \leftarrow 0$
3. **repetă**
4. $\quad t \leftarrow t + 1$
5. $\quad$ **pentru fiecare** $\theta_j \in \Theta$
6. $\quad\quad \theta_j^{(t)} = \theta_j^{(t-1)} - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h(x^{(i)}) - y^{(i)} \right) x^{(i)}$
7. **până când** $|f(\Theta^{(t)}) - f(\Theta^{(t-1)})| < \epsilon$
8. **întoarce** $\Theta^{(t)}$

- Metoda nu garantează găsirea valorii optime globale dacă funcția conține minime locale și/sau puncte șa
- Pe măsură ce ne apropiem de minimul local, se vor face pași din ce în ce mai mici
- Regimul de învățare este de tip batch în care utilizăm toate exemplele de învățare
- Viteza de învățare depinde de α (rata de învățare)
  - Dacă α este prea mic, algoritmul va converge încet
  - Dacă α este prea mare, algoritmul poate să nu găsească minimul
- Există criterii de convergență echivalente pentru algoritm (e.g. $|\Theta^{(t)} - \Theta^{(t-1)}| < \epsilon$)
- În practică, pentru problemele foarte mari (multe atribute, număr mare de exemple, etc.) putem impune o limită și pentru t

- Pentru o convergență rapidă a algoritmului de scădere a gradientului trebuie ca valorile de atribute să fie de același ordin
- Exemple:
  - Suprafață teren casă: $(0, 5000] m^2$
  - Număr camere: $[1, 7]$
  - Preț: $[10000, 1000000]$
- Aplicăm metode de re-scalare a atributelor:
  - Normalizare în intervalul $[a, b]$: $x_{normalizat} = (b - a)\frac{x - min(x)}{max(x) - min(x)} + a$
  - Standardizare (distribuție normală centrată în 0): $z = \frac{X - \mu}{\sigma}$
- În cazul în care nu rescalăm atributele se ajunge foarte greu sau deloc la convergență

Arbori de regresie

- Arborii de regresie sunt similari cu arborii de decizie, cu deosebirea că frunzele returnează valori continue în loc de clase
- Modelele de regresie liniară sunt modele globale, în sensul că o singură formulă predictivă este utilizată pentru tot spațiul în care operăm
- Dacă interacțiunea atributelor între ele și/sau a atributelor cu ieșirea nu este una liniară, predicția cu un singur model global este cel puțin problematică
- O soluție ar fi să partiționăm spațiul în regiuni mai mici și să considerăm exemplele doar din regiunea respectivă

Arbori de regresie

- Arborii de regresie partiționează spațiul recursiv, în mod *top-down* până la porțiuni suficient de mici unde se pot aplica modele simple
- Similar cu arborii de decizie, în fiecare nod testăm după un atribut a cărui valoare ne indică pe ce cale să coborâm în arbore
- Frunzele corespund unor vecinătăți din spațiul de intrare unde exemplele sunt similare ca ieșiri
- Astfel, arborii de decizie sunt un fel de nearest-neighbour adaptiv

Arbori de regresie

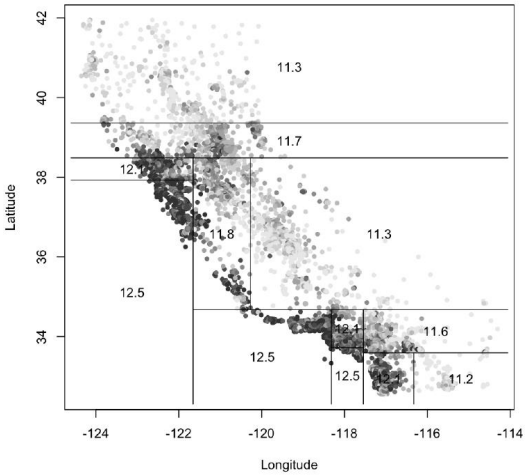
- O frunză conține un subset de c exemple: $(x^1, y^1), (x^2, y^2), ..., (x^c, y^c)$
- Astfel, pentru fiecare frunză putem folosi un model simplu:

$$y^{estimat} = \frac{1}{c} \sum_{i=1}^{n} y^i$$

- Prin urmare, flexibilitatea și eficiența (deci complexitatea) unui arbore este dată de câte frunze are

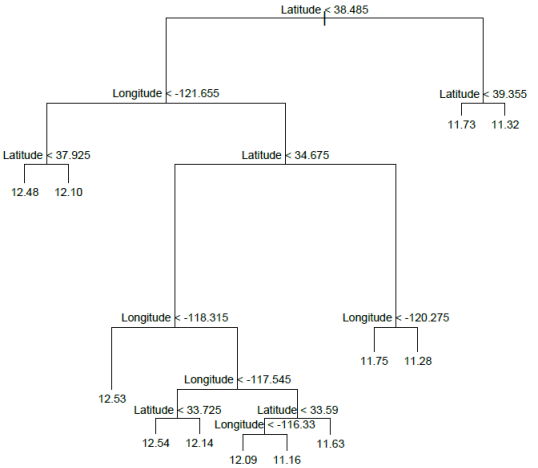
Arbori de regresie : Complexitate

Estimare cost locuințe în funcție de poziționare.



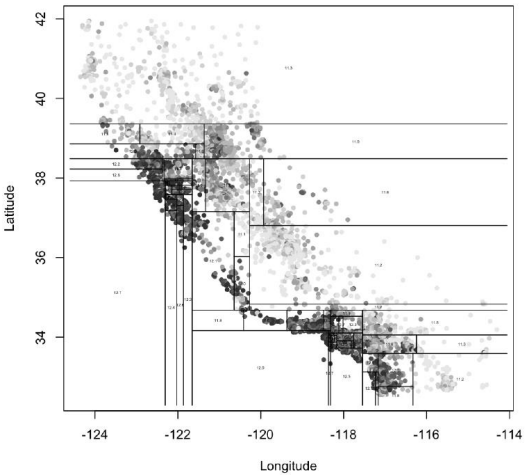
Arbori de regresie : Complexitate

Arbore de regresie pentru estimare cost locuințe în funcție de poziționare.



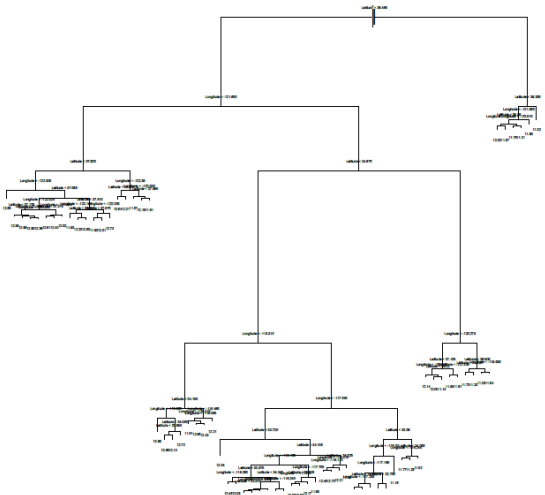
Arbori de regresie : Complexitate

Estimare (mai detaliată) cost locuințe în funcție de poziționare.



Arbori de regresie : Complexitate

Arbore de regresie pentru estimare (mai detaliată) cost locuințe în funcție de poziționare.



Arbori de regresie : Construcție

- Cât de mult creștem arborele de regresie?
- Problemele sunt similare cu cele ale arborelui de decizie
- Cum creștem arborele de regresie?
- Adăugarea unui nod trebuie să reducă eroarea
- Utilizăm funcția sumă reziduală pătratică (eng. RSS - Residual Sum of Squares):

$$RSS = \sum_{k \in X_{x_j \leq c}} (y_k - \bar{y}_{X_j \leq c})^2 + \sum_{k \in X_{x_j > c}} (y_k - \bar{y}_{X_j > c})^2$$

- Căutăm valoarea de tăietură c care împarte setul de date din nodul curent în două subseturi disjuncte, $X_{x_j \leq c}$ și $X_{x_j > c}$, astfel încât RSS să fie minim:

$$\underset{c}{argmin}\ RSS(X_j, c)$$

Arbori de regresie : Construcție

- Problemă: Găsirea celei mai bune partiționări binare folosind *RSS* nu este tractabil computațional
- Soluție: Utilizăm tehnici greedy de generare
- Plecând de la setul de date complet, considerăm toate atributele X_j după care putem împărți (unele pot fi discrete, altele continue)
- Pentru atributele discrete generăm atâția sub-arbori câte valori distincte avem (exact ca la arborii de decizie)
- Pentru atributele continue sortăm ascendent valorile unice și obținem posibile puncte c de tăietură folosind mediile valorilor alăturate
  - Exemplu: Pentru valorile unice sortate ascendent $\{10, 12, 13, 44, 50\}$ vom avea ca posibile puncte de tăietură mulțimea $\{11, 12.5, 28.5, 47\}$
- Căutăm valoarea de tăietură c care minimizează funcția *RSS*

Arbori de regresie : Criterii de oprire

- Care sunt criteriile de oprire?
  - Nodul conține mai puțin de numărul minim de puncte
  - Prin selecția unui atribut, eroarea nu scade cu mai mult de un prag minim
- Observație: pe o cale de la rădăcină către o frunză un atribut poate apărea de mai multe ori (dar cu intervale de valori diferite pentru fii pe care îi generează)
- Chiar și cu aceste condiții de oprire, încă există posibilitatea de generare a unor arbori de regresie de dimensiuni mari
- Există tehnici de tăiere a sub-arborilor care reduc dimensiunea arborelui de regresie construit (complet) la una acceptabilă din punct de vedere al complexității, păstrând însă *RSS* mic

Mulțumesc!



Feedbackul este binevenit!